

PSP - PostgreSQL Vacuum

Context

Since the database is being migrated to Postgres, it is essential to periodically monitor and analyze vacuum operations, and refine them accordingly. This document sheds light on the process of monitoring, analyzing, and offering recommendations for vacuum operations in Postgres.

Principles

- Autovacuum needs to run at regular intervals without taking much time & resources.
- Prevent aggressive vacuum on multiple large tables to avoid performance impact.
- Prevent vacuum operations impacting database stability.

Data Collection & Analysis

Each database has its unique characteristics in terms of its size, transaction rate, and traffic pattern. Therefore, it is advisable to gather sufficient information about the database before modifying the parameters or implementing a manual vacuum or analyze regime. Collect below data for analysis for large and high transactional tables (It can be collected for all tables as well for reference) to start with.

- Insert, update & delete operations per day/week
- Total number of rows
- Number of dead tuples
- When was last autovacuum and autoanalyze happened
- The time taken by autovacuum for each table
- Any Warnings/error/delays about tables not being vacuumed
- Any performance issues for queries because of statistics

Analyze above collected data to come up with number to set thresholds for autovacuum at table level. [Reference](#)

Recommendations

Database level

Note: If postgres is built via IPS(Intuit Paved Services) then few of below setting might be coming from IPS.

- Enable auto vacuum by setting autovacuum db parameter to 1/on.
- Enable auto vacuum logging.
- Monitor IOPS/IO latency caused by vacuum operation and adjust autovacuum_max_workers parameter accordingly

Tune Autovacuum threshold

Table level

- Monitor and Analyze dead tuples for large and busy tables. If its high number(>50000/day)
- set autovacuum_vacuum_scale_factor to 0 and set autovacuum_vacuum_threshold to number arrived from above step (for eg. 500000). Table will be autovacuumed when its number of dead rows is more than 500000.

Tune Autoanalyze threshold

- Monitor and Analyze dead tuples for large and busy tables. If its high number(>50000/day)
- set autovacuum_vacuum_scale_factor to 0 and set autovacuum_vacuum_threshold to number arrived from above step (for eg. 500000). Table will be autovacuumed when its number of dead rows is more than 500000.

Note:

Design level considerations

Below considerations would also help in vacuum operations without much impact by ensuring less amount work to do for tables.

- Consider table for partitioning whose size is >100G.
- Consider tables sizes and growth rate (estimated/projected next 3 year and 5 year)
- Try to Keep individual partition size ~50G

Monitoring

- Setup MaximumUsedTransactionIDs warning alert to 5% more than value of autovacuum_freeze_max_age
- Setup MaximumUsedTransactionIDs critical alert to 10% more than value of autovacuum_freeze_max_age

- Monitor vacuum frequency for all tables
- Create dashboard to have all DML metrics for all tables
- Set up monitoring for slow running queries
 - Queries whose execution time is more than 1/5 mins(depends upon OLTP performance SLA)
 - If there is batch workload then queries whose execution time is more than 30/40 mins(depends upon OLAP performance SLA)
 - Analyze queries if its because of stale statistics. This also could lead to autovacuum not doing its job and autovacuum getting delayed.